Analyses: Feature Value Distributions

Description

This file provides functionality for plotting feature value distributions in density plots. It also contains code for PCA analyses given the values of the four statistical features (entropy rate, unigram entropy, TTR, and repetition rate).

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"

## [1] "x86_64-pc-linux-gnu"
```

Load libraries

If the libraries are not installed yet, you need to install them using, for example, the command: install.packages("ggplot2").

```
#library(devtools)
#install_github("vqv/ggbiplot")
library(ggbiplot)
```

```
## Loading required package: ggplot2
```

```
library(ggplot2)
library(ggrepel)
library(ggExtra)
library(ggpubr)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(ggridges)
library(GGally)
```

```
## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg   ggplot2
```

```r
library(hexbin)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v stringr   1.5.1
## v lubridate 1.9.3     v tibble    3.2.1
## v purrr     1.0.2     v tidyr     1.3.1
## v readr     2.1.5
```

```
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Give the package versions.

```
## lubridate   forcats   stringr     purrr     readr     tidyr    tibble tidyverse
##   "1.9.3"   "1.0.0"   "1.5.1"   "1.0.2"   "2.1.5"   "1.3.1"   "3.2.1"   "2.0.0"
##    hexbin    GGally  ggridges     dplyr    ggpubr   ggExtra   ggrepel  ggbiplot
##  "1.28.3"   "2.2.1"   "0.5.6"   "1.1.4"   "0.6.0"  "0.10.1"   "0.9.5"   "0.6.2"
##   ggplot2
##   "3.5.1"
```

Load Data

Load csv file with quantitative feature values for sign sequences per object (paleolithic signs) and tablet (cuneiform). A comparative sample of feature values for languages across the world is derived from the Text Data Diversity (TeDDi) sample (Moran et al 2022).

```r
estimations.df <- read.csv("~/Github/PaleoSigns/results/features/features_combined.csv")
head(estimations.df)
```

```
##        id   subcorpus           sequence num.tokens num.types huni.chars
## 1 bst0001 Aurignacien vvvvvvvvvvvvvvvvvv         18         1  0.0000000
## 2 gkl0001 Aurignacien       ||| //////_V         12         5  1.8962406
## 3 gkl0002 Aurignacien              vvvvv          5         1  0.0000000
## 4 gkl0003 Aurignacien          vvvv vvvv          9         2  0.5032583
## 5 gkl0004 Aurignacien     vvvvvvvvvvvvvv         14         1  0.0000000
## 6 gkl0005 Aurignacien                |||          3         1  0.0000000
##   hrate.chars  ttr.chars  rm.chars
## 1   0.6171397 0.05555556 1.0000000
## 2   1.5295563 0.41666667 0.6363636
## 3   0.5711903 0.20000000 1.0000000
## 4   0.9060047 0.22222222 0.7500000
## 5   0.6379550 0.07142857 1.0000000
## 6   0.4308271 0.33333333 1.0000000
```

Preprocessing

```r
# select only sign sequences of certain length
#estimations.df <- estimations.df[estimations.df$num.tokens > 10 &
#                                   estimations.df$num.tokens < 50,]

# select only certain subcorpora for plotting
estimations.df.selected <- estimations.df[estimations.df$subcorpus == "Aurignacien"|
                                          estimations.df$subcorpus == "Cuneiform (Uruk V)"|
                                          estimations.df$subcorpus == "Cuneiform (Uruk IV)"|
                                          estimations.df$subcorpus == "Cuneiform (Uruk III)"|
                                          estimations.df$subcorpus == "TeDDi", ]
```

Simple Stats

```r
# number of sequences
nrow(estimations.df.selected)
```

```
## [1] 3076
```

```r
# number of rows (i.e. sequences) per subcorpus
table(estimations.df.selected$subcorpus)
```

```
##
##          Aurignacien Cuneiform (Uruk III)  Cuneiform (Uruk IV)
##                  211                  855                  859
##   Cuneiform (Uruk V)                TeDDi
##                  111                 1040
```

```r
# get mean values of lengths in tokens per subcorpus
aggregate(estimations.df.selected$num.tokens~
            estimations.df.selected$subcorpus, FUN = mean)
```

```
##   estimations.df.selected$subcorpus estimations.df.selected$num.tokens
## 1                       Aurignacien                          14.218009
## 2              Cuneiform (Uruk III)                          32.240936
## 3               Cuneiform (Uruk IV)                          21.770664
## 4                Cuneiform (Uruk V)                           8.648649
## 5                             TeDDi                          39.730769
```

```r
# get median values of lengths in tokens per subcorpus
aggregate(estimations.df.selected$num.tokens~
            estimations.df.selected$subcorpus, FUN = median)
```

```
##   estimations.df.selected$subcorpus estimations.df.selected$num.tokens
## 1                       Aurignacien                                  8
## 2              Cuneiform (Uruk III)                                 17
## 3               Cuneiform (Uruk IV)                                 13
## 4                Cuneiform (Uruk V)                                  7
## 5                             TeDDi                                 28
```

Density Plots

Length

Density plot for lengths in number of tokens.

```
length.plot <- ggplot(estimations.df.selected,
                      aes(x = num.tokens, y = fct_reorder(subcorpus, num.tokens),
                          fill = subcorpus)) +
  # add density ridges with quantiles = 2, i.e. median as a line
  geom_density_ridges(alpha = 0.5, quantile_lines = TRUE, quantiles = 2, scale = 1) +
  geom_point(shape = 124, alpha = 0.3, size = 3) +
  # limit the x-range
  xlim(0, 100) +
  xlab("Number of tokens") +
  ylab("Subcorpus") +
  theme(legend.position = "none")
#length.plot
```

Safe complete figure to file

```
#ggsave("~/Github/PaleoSigns/figures/lengthPlot_chars.pdf",
#       length.plot, dpi = 300, width = 5, height = 5, device = cairo_pdf)
```

Get subset of data points to plot labels.

```
# choose subset of data points for illustration
selected.ids <- c("vhc0017", "P000842", "P001084", "P000196",
                  "639415", "5410574", "9589824", "17247600",
                  "14002638", "5021755")
estimations.df.illustration <- estimations.df.selected[estimations.df.selected$id %in% selected.ids, ]
```

Illustration Plots

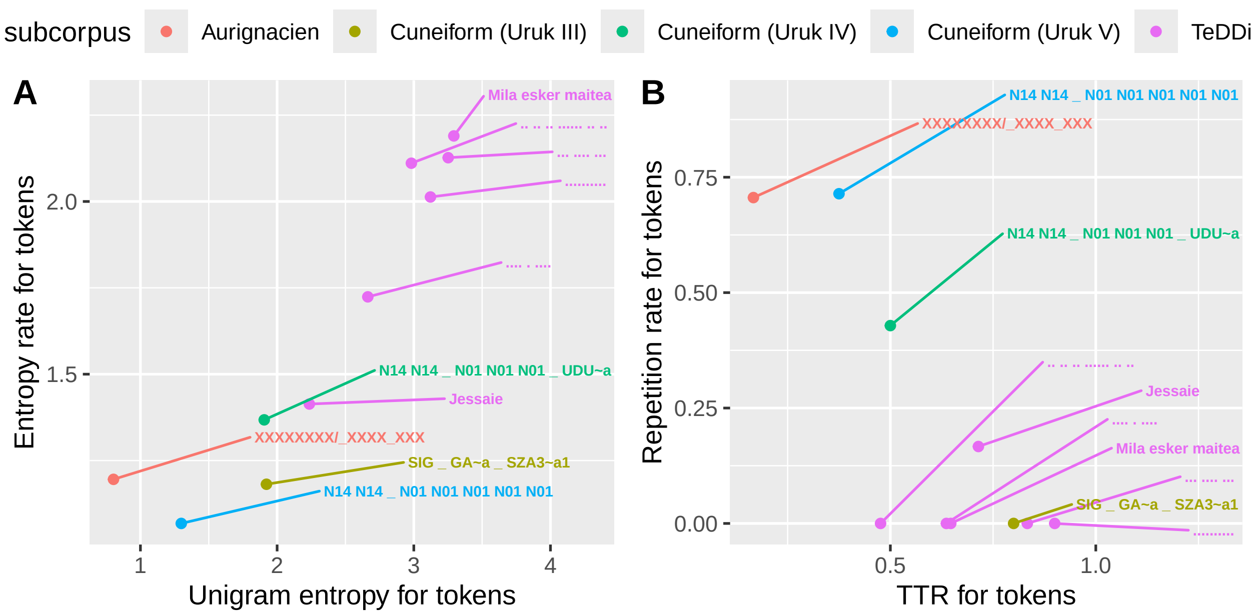
```
# Unigram entropy vs. Entropy rate
huni.hrate.plot <- ggplot(estimations.df.illustration, aes(x = huni.chars,
                                                           y =  hrate.chars, color = subcorpus)) +
  geom_point() +
  labs(x = "Unigram entropy for tokens", y = "Entropy rate for tokens") +
  geom_text_repel(aes(label = sequence),
                  size = 2,
                  hjust = 0,
                  direction = 'y',
                  fontface = "bold",
                  nudge_x = 1,
                  nudge_y = 0.1,
                  max.overlaps = Inf,
                  show.legend = F)

# TTR vs. Repetition rate
ttr.rm.plot <- ggplot(estimations.df.illustration, aes(x = ttr.chars,
                                                       y =  rm.chars, color = subcorpus)) +
  geom_point() +
```

```
  labs(x = "TTR for tokens", y = "Repetition rate for tokens") +
  geom_text_repel(aes(label = sequence),
                  size = 2,
                  hjust = 0,
                  direction = 'y',
                  fontface = "bold",
                  nudge_x = 0.4,
                  nudge_y = 0.2,
                  max.overlaps = Inf,
                  show.legend = F)
```

Combine figures.

```
combined.plot.illustration <- ggarrange(huni.hrate.plot, ttr.rm.plot, ncol = 2,
                                         labels = c("A", "B"), common.legend = T, legend = "top")
combined.plot.illustration
```



Safe complete figure to file

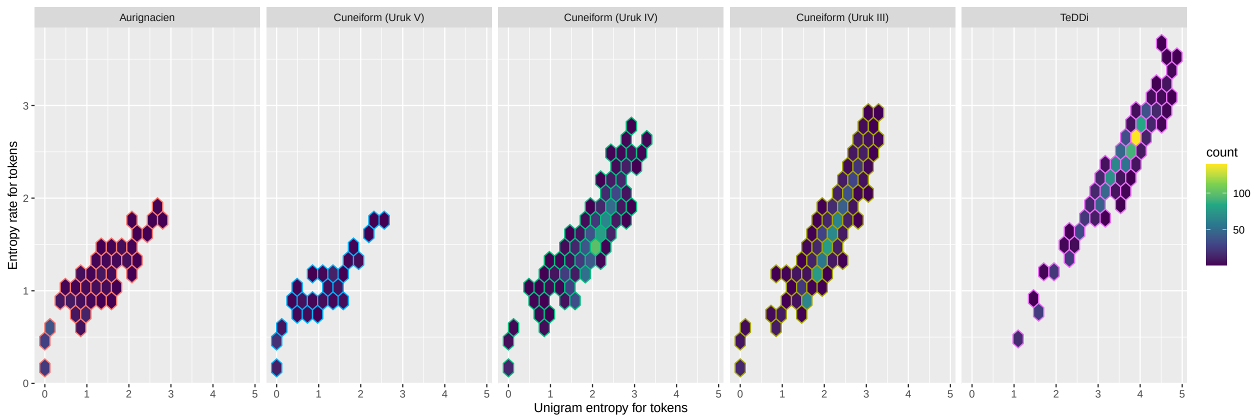
```
ggsave("~/Github/PaleoSigns/figures/combinedPlot_illustration.pdf",
       combined.plot.illustration, dpi = 300, width = 7, height = 3.5, device = cairo_pdf)
```

Full Feature Distribution Plots

Entropy rate vs. Unigram Entropy

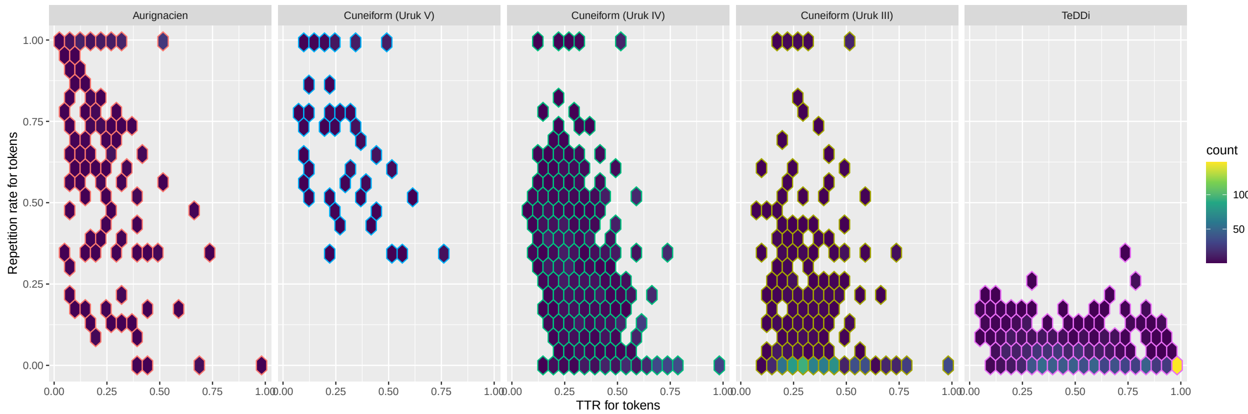
```
huni.hrate.chars.plot <- ggplot(estimations.df.selected, aes(x = huni.chars, y =  hrate.chars,
                                                              color = subcorpus)) +
  scale_colour_discrete(guide = "none") +
  geom_hex(bins = 20) +
  scale_fill_continuous(type = "viridis") +
  labs(x = "Unigram entropy for tokens", y = "Entropy rate for tokens") +
  # geom_label_repel(data = labels.df,
```

```
#                   aes(label = sequence),
#                   size = 2,
#                   hjust = 0,
#                   direction = 'y',
#                   fontface = "bold",
#                   nudge_x = 2,
#                   max.overlaps = Inf,
#                   show.legend = FALSE) +
#guides(size = guide_legend(title = "No. of chars")) +
facet_grid(~factor(subcorpus, levels = c('Aurignacien', 'Cuneiform (Uruk V)',
                                         'Cuneiform (Uruk IV)', 'Cuneiform (Uruk III)',
                                         'TeDDi')))
huni.hrate.chars.plot
```



TTR vs. Repetition Rate

```
ttr.rm.chars.plot <- ggplot(estimations.df.selected, aes(x = ttr.chars, y = rm.chars,
                                  colour = subcorpus)) +
  scale_colour_discrete(guide = "none") +
  geom_hex(bins = 20) +
  scale_fill_continuous(type = "viridis") +
  labs(x = "TTR for tokens", y = "Repetition rate for tokens") +
  # geom_label_repel(data = labels.df,
  #                  aes(label = sequence),
  #                  size = 2,
  #                  hjust = 0,
  #                  direction = 'y',
  #                  fontface = "bold",
  #                  #nudge_x = 0.2,
  #                  max.overlaps = Inf,
  #                  show.legend = FALSE) +
  facet_grid(~factor(subcorpus, levels = c('Aurignacien', 'Cuneiform (Uruk V)',
                                           'Cuneiform (Uruk IV)', 'Cuneiform (Uruk III)',
                                           'TeDDi')))
ttr.rm.chars.plot
```



Combine figures.

```
combined.plot1 <- ggarrange(huni.hrate.chars.plot, ttr.rm.chars.plot, ncol = 1,
                            labels = c("A", "B"))
```

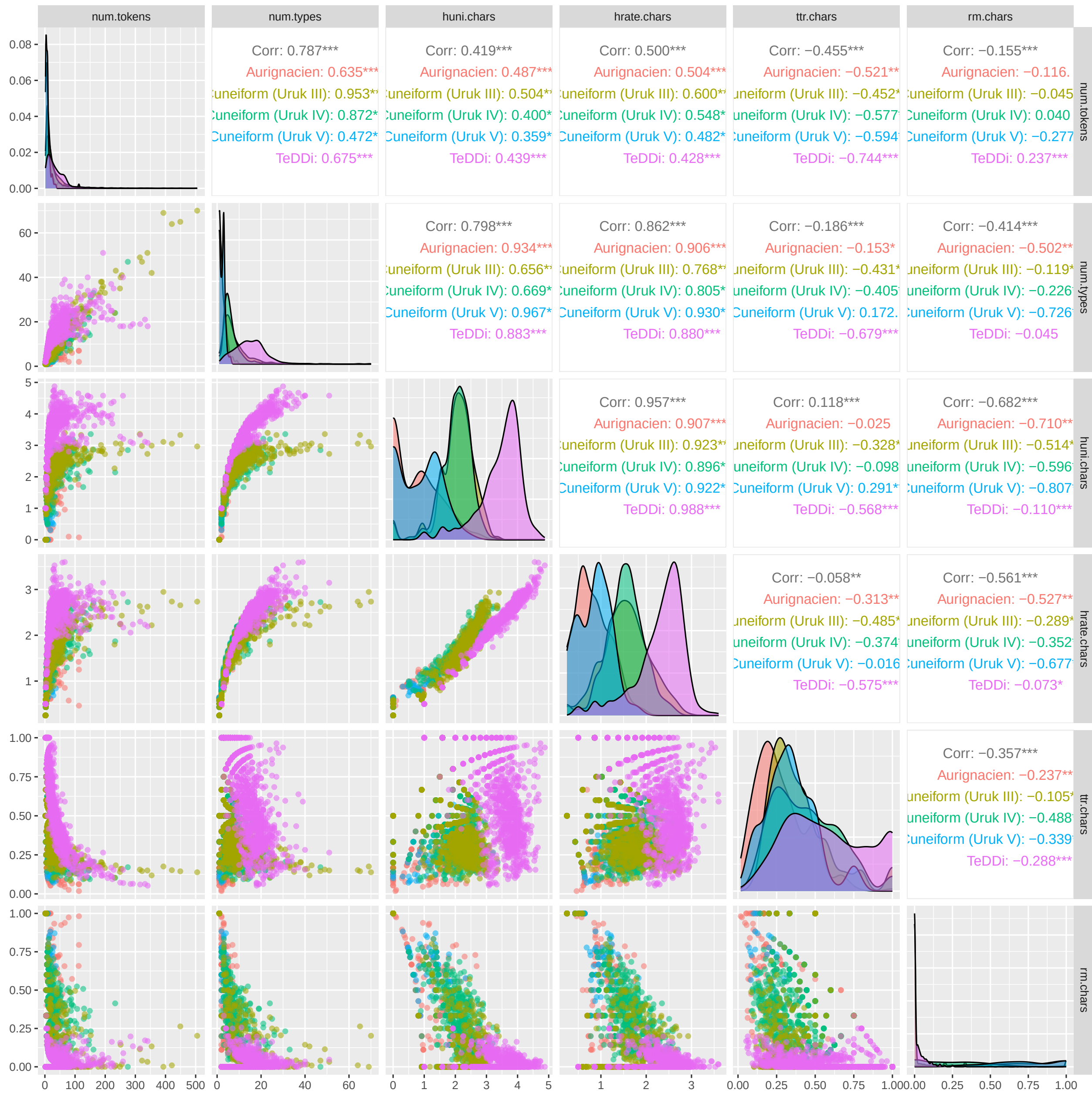
Safe complete figure to file

```
ggsave("~/Github/PaleoSigns/figures/combinedPlot_chars.pdf",
       combined.plot1, dpi = 300, width = 12, height = 6, device = cairo_pdf)
```

Correlation Plot

Correlation plots for the statistical feature values.

```
cor.plot <- ggpairs(estimations.df.selected,
                    columns = c("num.tokens", "num.types",
                                "huni.chars", "hrate.chars",
                                "ttr.chars","rm.chars"),
        aes(color = subcorpus,
            alpha = 0.5))
cor.plot
```



Safe complete figure to file.

```r
ggsave("~/Github/PaleoSigns/figures/correlation_plot.pdf",
       cor.plot, dpi = 300, width = 12, height = 12, device = cairo_pdf)
```

Principal Component Analysis (PCA)

Since we have overall four dimensions of statistical values derived from the sign sequences (unigram entropy, entropy rate, TTR, repetition rate), we can first use a principal component analysis to reduce the number of dimensions to the first two principal components. This allows us to see the clustering of data points in two dimensional space.

Note that we here use PCA as a data visualization and exploration tool, rather than in an inferential sense (cf. Jolliffe and Cadima, 2016, p. 2). For drawing statistical inferences we rather employ statistical analyses of classification outputs (see analyses_Classification_performance.Rmd).

Compute principal components.

```
pca <- prcomp(estimations.df.selected[c("huni.chars",
                                        "hrate.chars",
                                        "ttr.chars",
                                        "rm.chars")],
             center = TRUE,
             scale. = TRUE)
print(pca)
```

```
## Standard deviations (1, .., p=4):
## [1] 1.5852893 1.0554699 0.5912949 0.1523530
##
## Rotation (n x k) = (4 x 4):
##                    PC1        PC2        PC3         PC4
## huni.chars   0.6110194 -0.1498861 -0.2635939 -0.73123708
## hrate.chars  0.5747523 -0.3273557 -0.3383643  0.66933374
## ttr.chars    0.1492465  0.8897060 -0.4210577  0.09412298
## rm.chars    -0.5234888 -0.2807057 -0.7992110 -0.09179079
```

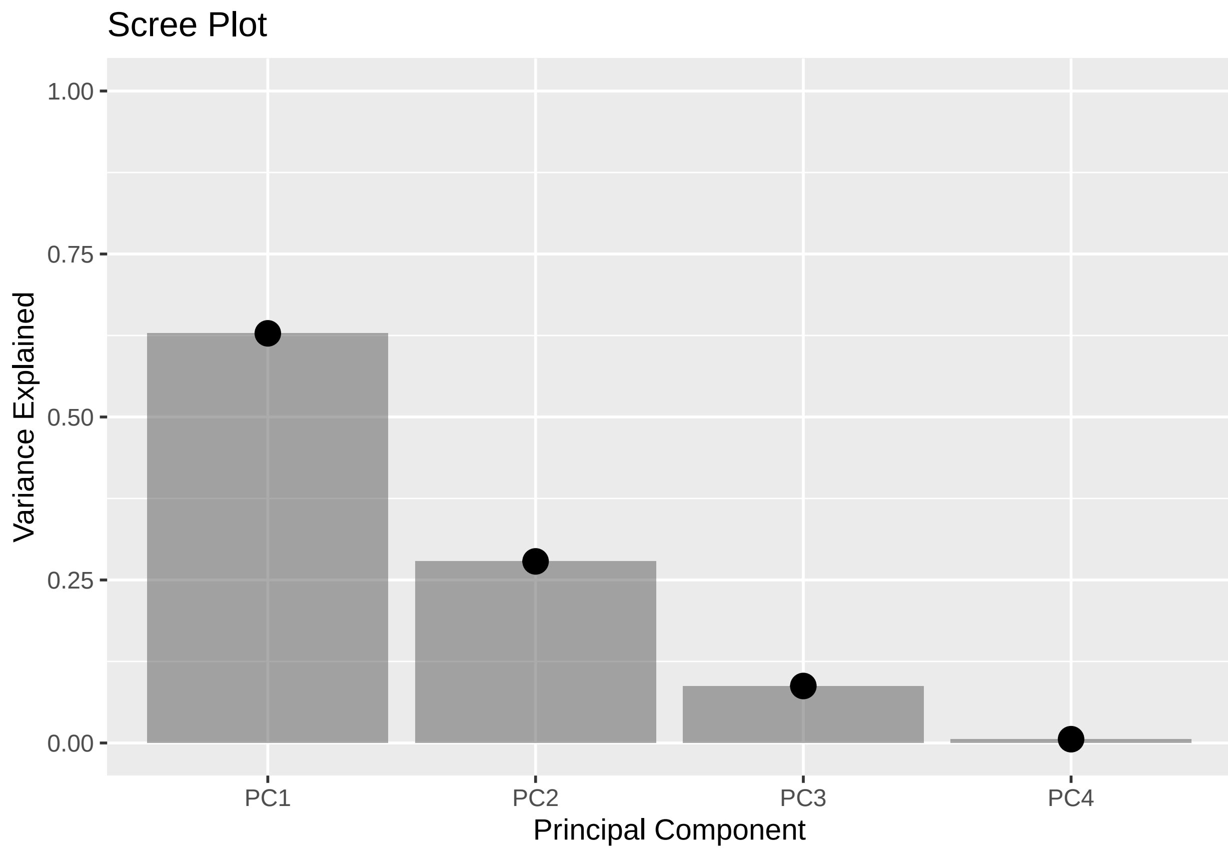
```
summary(pca)
```

```
## Importance of components:
##                           PC1    PC2     PC3    PC4
## Standard deviation     1.5853 1.0555 0.59129 0.1524
## Proportion of Variance 0.6283 0.2785 0.08741 0.0058
## Cumulative Proportion  0.6283 0.9068 0.99420 1.0000
```

Screeplot with loadings of principal components.

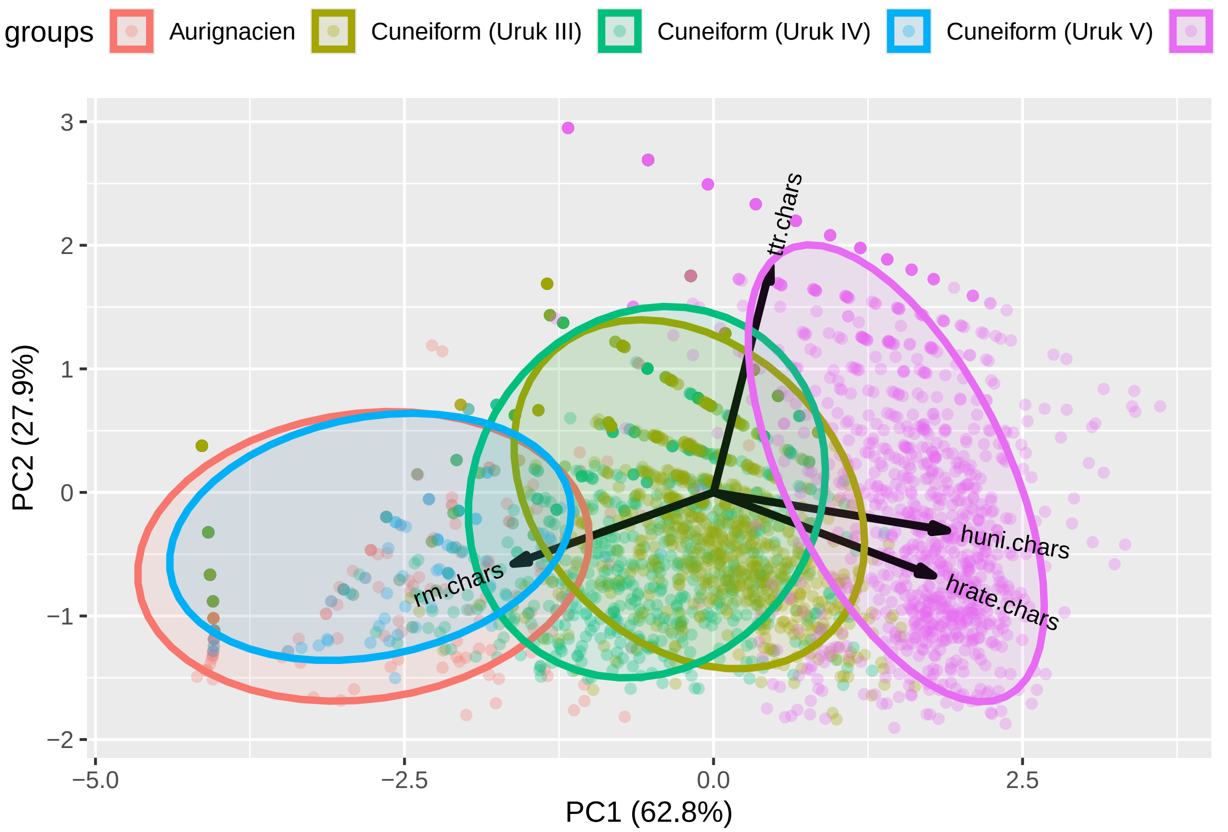
```
# compute total variance
variance = pca $sdev^2 / sum(pca $sdev^2)
PCs <- paste("PC", c(1:4), sep = "")
pca.df <- data.frame(PCs, variance)

# Scree plot
ggplot(pca.df, aes(x = PCs, y = variance)) +
  geom_col(alpha = 0.5) +
  geom_point(size = 4)+
  xlab("Principal Component") +
  ylab("Variance Explained") +
  ggtitle("Scree Plot") +
  ylim(0, 1)
```



Bi-plot with ggplot.

```
biplot <- ggbiplot(pca,
              obs.scale = 1,
              var.scale = 1,
              groups = estimations.df.selected$subcorpus,
              ellipse = TRUE,
              ellipse.alpha = 0.1,
              circle = FALSE,
              #var.axes = T,
              #ellipse.prob = 0.68, # NULL = 0.68
              alpha = 0.3) +
  theme(legend.position = 'top')
print(biplot)
```



Interpretation: The first two principal components together preserve c. 90% of the original variance in the four variables. PC1 mainly reflects the opposition between repetition rate and entropy (both unigram and rate). Unigram entropy and entropy rate are highly positively correlated (similar vector directions) and have a positive loading on PC1. Modern writing systems fall into the high entropy area, while proto-cuneiform systems tend to have high repetition rates, and low entropies similar to the Aurignacian sequences of signs. TTR is mostly orthogonal to the other three measures and loads mainly on PC2. Aurignacian and Uruk V signs tend to have low TTR, while modern writing systems display a wide range of TTRs (depending on the character inventories).

The normal data ellipse for the Aurignacian sign sequences fully overlaps with the one for the earliest Proto-Cuneiform period (Uruk V), whereas the later two Proto-Cuneiform periods (Uruk IV and Uruk III) start to "move closer" and overlap with the TeDDi (modern writing) ellipse.

Safe complete figure to file

```
#ggsave("~/Github/PaleoSigns/figures/pca_biplot.pdf",
#       biplot, dpi = 300, device = cairo_pdf)
```

Combine figures.

```
combined.plot2 <- ggarrange(length.plot, biplot, ncol = 2,
                            widths = c(1,2), labels = c("C", "D"))
```

```
## Picking joint bandwidth of 3.18
```

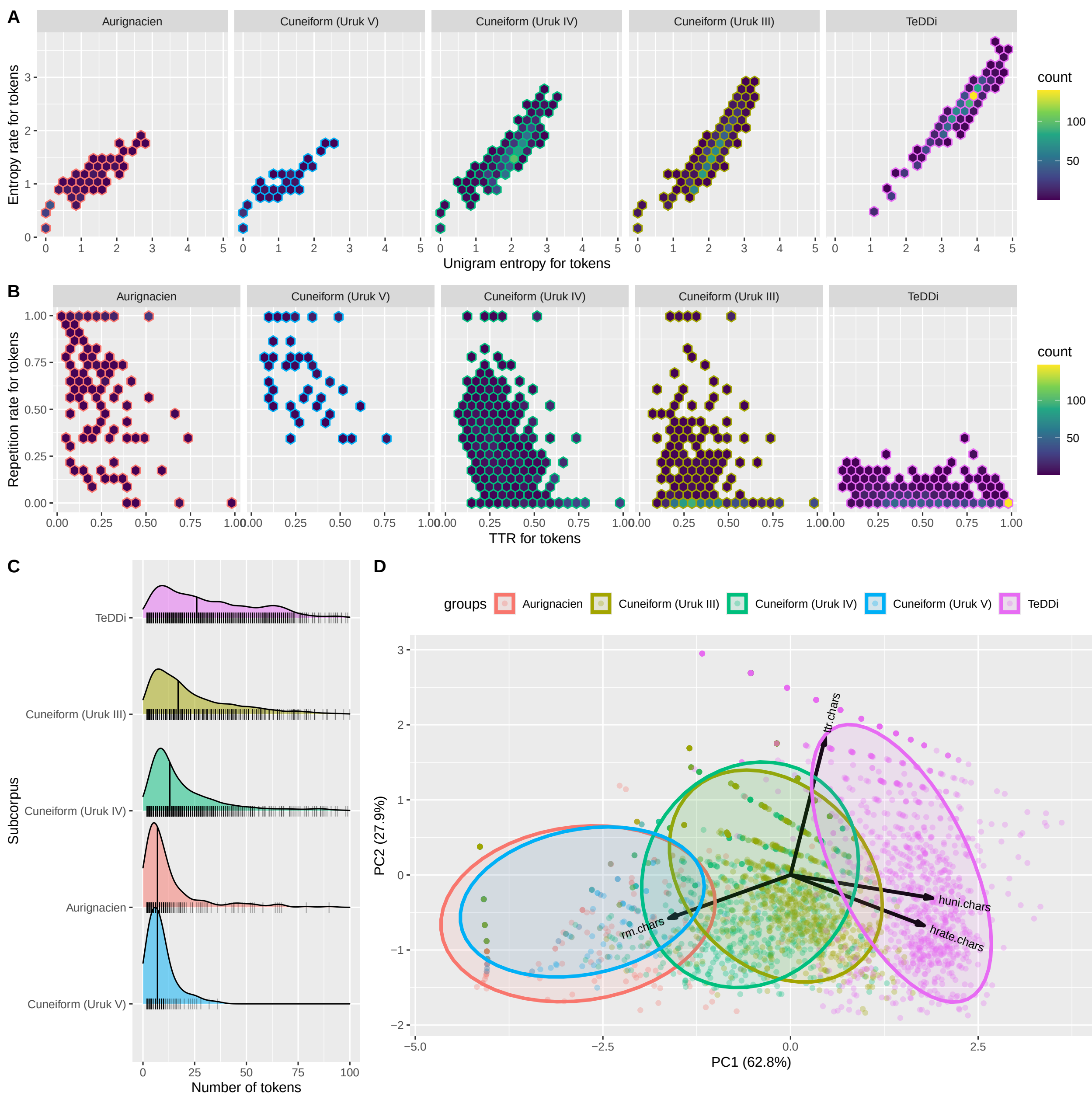
```
## Warning: Removed 131 rows containing non-finite outside the scale range
## (`stat_density_ridges()`).
```

```
## Warning: Removed 131 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

Final Combination of Plot Panels

Combine figures.

```
final.plot <- ggarrange(combined.plot1,
                        combined.plot2, ncol = 1, nrow = 2)
final.plot
```



Safe complete figure to file

```
ggsave("~/Github/PaleoSigns/figures/finalPlot_chars.pdf",
       final.plot, dpi = 300, width = 12, height = 12, device = cairo_pdf)
```

Package References


```
## Vu VQ, Friendly M (2024). _ggbiplot: A Grammar of Graphics
## Implementation of Biplots_. R package version 0.6.2,
## <https://CRAN.R-project.org/package=ggbiplot>.

## Wickham H, François R, Henry L, Müller K, Vaughan D (2023). _dplyr: A
## Grammar of Data Manipulation_. R package version 1.1.4,
## <https://CRAN.R-project.org/package=dplyr>.

## Carr D, Lewin-Koh pbN, Maechler M, Sarkar ccolfwbD (2023). _hexbin:
## Hexagonal Binning Routines_. R package version 1.28.3,
## <https://CRAN.R-project.org/package=hexbin>.

## Wickham H, Averick M, Bryan J, Chang W, McGowan LD, François R,
## Grolemund G, Hayes A, Henry L, Hester J, Kuhn M, Pedersen TL, Miller E,
## Bache SM, Müller K, Ooms J, Robinson D, Seidel DP, Spinu V, Takahashi
## K, Vaughan D, Wilke C, Woo K, Yutani H (2019). "Welcome to the
## tidyverse." _Journal of Open Source Software_, *4*(43), 1686.
## doi:10.21105/joss.01686 <https://doi.org/10.21105/joss.01686>.

## Slowikowski K (2024). _ggrepel: Automatically Position Non-Overlapping
## Text Labels with 'ggplot2'_. R package version 0.9.5,
## <https://CRAN.R-project.org/package=ggrepel>.

## Attali D, Baker C (2023). _ggExtra: Add Marginal Histograms to
## 'ggplot2', and More 'ggplot2' Enhancements_. R package version 0.10.1,
## <https://CRAN.R-project.org/package=ggExtra>.

## Kassambara A (2023). _ggpubr: 'ggplot2' Based Publication Ready Plots_.
## R package version 0.6.0, <https://CRAN.R-project.org/package=ggpubr>.

## Wilke C (2024). _ggridges: Ridgeline Plots in 'ggplot2'_. R package
## version 0.5.6, <https://CRAN.R-project.org/package=ggridges>.

## Schloerke B, Cook D, Larmarange J, Briatte F, Marbach M, Thoen E,
## Elberg A, Crowley J (2024). _GGally: Extension to 'ggplot2'_. R package
## version 2.2.1, <https://CRAN.R-project.org/package=GGally>.
```


Literature References

Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: a review and recent developments. Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences, 374(2065), 20150202.

Moran, S., Bentz, C., Gutierrez-Vasques, X., Pelloni, O., & Samardzic, T. (2022, June). TeDDi sample: text data diversity sample for language comparison and multilingual NLP. In Proceedings of the Thirteenth Language Resources and Evaluation Conference (pp. 1150-1158).